

ConcurrentDictionary Collection Class in C# with Examples

In this article, I am going to discuss the **ConcurrentDictionary Collection Class in C#** with Examples. Please read our previous article where we discussed the [Concurrent Collection in C#](#) with Examples. The ConcurrentDictionary collection was introduced in .NET 4.0 and it belongs to System.Collections.Concurrent namespace. As part of this article, we are going to discuss the following pointers.

1. [What is ConcurrentDictionary in C#?](#)
2. [How to create a ConcurrentDictionary Collection in C#?](#)
3. [How to Add Elements into a ConcurrentDictionary <TKey, TValue> Collection in C#?](#)
4. [How to access a ConcurrentDictionary <TKey, TValue> Collection in C#?](#)
5. [How to Check the Availability of a key/value pair in ConcurrentDictionary Collection in C#?](#)
6. [How to Remove Elements from ConcurrentDictionary Collection in C#?](#)
7. [Understanding TryUpdate Method of ConcurrentDictionary Collection Class in C#](#)
8. [Understanding AddOrUpdate Methods of ConcurrentDictionary Collection Class in C#](#)
9. [Understanding GetOrAdd Methods of ConcurrentDictionary Collection Class in C#](#)
10. [Understanding TryGetValue Method of ConcurrentDictionary Collection Class in C#](#)
11. [ConcurrentDictionary Collection with Complex Type in C#](#)
12. [How to Get all the Keys and Values of a ConcurrentDictionary in C#?](#)

What is ConcurrentDictionary in C#?

The **ConcurrentDictionary<TKey, TValue>** represents a thread-safe collection of key/value pairs that can be accessed by multiple threads concurrently.

The ConcurrentDictionary<TKey, TValue> is a Concurrent Collection that stores the element in the form of Key-Value Pairs. The working of the **ConcurrentDictionary<TKey, TValue>** is very much similar to the working of the [Generic Dictionary<TKey, TValue>](#) collection class. The only difference is that Generic Dictionary is not thread-safe whereas the ConcurrentDictionary is thread-safe.

It is also possible to use the Dictionary Class instead of ConcurrentDictionary with multiple threads, but in that case, as a developer, we need to use locks explicitly to provide thread safety which is always time-consuming and error-prone. So, the ideal choice is to use ConcurrentDictionary instead of Dictionary in a multi-threaded environment.

The ConcurrentDictionary Collection class internally manages the locking which gives us an easy interface to Add/Update/Remove items. This class provides different methods to Add, Retrieve, Update and Remove items. At the end of this article, you will understand all those methods with examples.

Points to Remember while Working with ConcurrentDictionary Collection in C#

Following are some important points that you need to keep in mind while working with ConcurrentDictionary Class in C#.

1. In ConcurrentDictionary<TKey, TValue> Collection, the key cannot be null, but the value can be null if its type TValue is a reference type.
2. Every key in the ConcurrentDictionary<TKey, TValue> Collection must be unique. Duplicate keys are not allowed. But if multiple threads try to add duplicate keys, then it will not allow and it will not throw any exception.

3. In `ConcurrentDictionary<TKey, TValue>` Collection, you can only store the same types of elements as it is generic and while creating the `ConcurrentDictionary` instance we need to specify the types for both key and values.
4. The capacity of a `ConcurrentDictionary` collection is the number of elements that the `ConcurrentDictionary` can hold.
5. The `ConcurrentDictionary<TKey, TValue>` collection is dynamic in nature means the size of the collection is automatically increased as we added items to the collection.

How to create a ConcurrentDictionary Collection in C#?

The `ConcurrentDictionary<TKey, TValue>` Collection class in C# provided the following constructors that we can use to create an instance of the `ConcurrentDictionary` collection class.

1. **`ConcurrentDictionary()`**: It initializes a new instance of the `ConcurrentDictionary` class that is empty, has the default concurrency level, has the default initial capacity, and uses the default comparer for the key type.
2. **`ConcurrentDictionary(IEnumerable<KeyValuePair<TKey, TValue>> collection)`**: It initializes a new instance of the `ConcurrentDictionary` class that contains elements copied from the specified `System.Collections.Generic.IEnumerable` has the default concurrency level, has the default initial capacity, and uses the default comparer for the key type.
3. **`ConcurrentDictionary(IEqualityComparer<TKey> comparer)`**: It initializes a new instance of the `ConcurrentDictionary` class that is empty, has the default concurrency level and capacity, and uses the specified `System.Collections.Generic.IEqualityComparer`.
4. **`ConcurrentDictionary(int concurrencyLevel, int capacity)`**: It initializes a new instance of the `ConcurrentDictionary` class that is empty, has the specified concurrency level and capacity, and uses the default comparer for the key type.
5. **`ConcurrentDictionary(IEnumerable<KeyValuePair<TKey, TValue>> collection, IEqualityComparer<TKey> comparer)`**: It initializes a new instance of the `ConcurrentDictionary` class that contains elements copied from the specified `System.Collections.Generic.IEnumerable` has the default concurrency level, has the default initial capacity, and uses the specified `System.Collections.Generic.IEqualityComparer`.
6. **`ConcurrentDictionary(int concurrencyLevel, IEnumerable<KeyValuePair<TKey, TValue>> collection, IEqualityComparer<TKey> comparer)`**: It initializes a new instance of the `ConcurrentDictionary` class that contains elements copied from the specified `System.Collections.Generic.IEnumerable`, and uses the specified `System.Collections.Generic.IEqualityComparer`.
7. **`ConcurrentDictionary(int concurrencyLevel, int capacity, IEqualityComparer<TKey> comparer)`**: It initializes a new instance of the `ConcurrentDictionary` class that is empty, has the specified concurrency level, has the specified initial capacity, and uses the specified `System.Collections.Generic.IEqualityComparer`.

Let's see how to create an instance of the `ConcurrentDictionary` collection class using the `ConcurrentDictionary()` constructor. The `ConcurrentDictionary()` constructor is used to create a new instance of the `ConcurrentDictionary` class that is empty, has the default concurrency level, has the default initial capacity, and uses the default comparer for the key type.

Step1:

As the `ConcurrentDictionary<TKey, TValue>` collection class belongs to `System.Collections.Concurrent` namespace, so first, we need to import the `System.Collections.Concurrent` namespace in our program is as follows:

```
using System.Collections.Concurrent;
```

Step2:

Next, we need to create an instance of the `ConcurrentDictionary<TKey, TValue>` class using the `ConcurrentDictionary()` constructor as follows:

```
ConcurrentDictionary<KeyDataType, ValuDataType> dictionary_name = new  
ConcurrentDictionary<KeyDataType, ValuDataType>();
```

How to Add Elements into a ConcurrentDictionary <TKey, TValue> Collection in C#?

Now, if you want to add elements i.e. a key/value pair into the `ConcurrentDictionary`, then you need to use the following `TryAdd ()` method of the `ConcurrentDictionary` Collection Class in C#.

1. **TryAdd(TKey key, TValue value):** The `TryAdd(TKey key, TValue value)` method is used to add an element with the specified key and value into the `ConcurrentDictionary` collection. Here, the parameter `key` specifies the key of the element to add and the parameter `value` specifies the value of the element to add. The value can be null for a reference type but the Key cannot be null. This method returns true if the key/value pair was added to the `ConcurrentDictionary` successfully; false if the key already exists. If the key is null, then it will throw `ArgumentNullException`. It will throw `OverflowException` if the dictionary already contains the maximum number of elements i.e. `System.Int32.MaxValue`.

For Example:

```
ConcurrentDictionary<string, string> dictionaryCountries = new ConcurrentDictionary<string, string>  
();  
dictionaryCountries.TryAdd("UK", "London, Manchester, Birmingham");  
dictionaryCountries.TryAdd("USA", "Chicago, New York, Washington");  
dictionaryCountries.TryAdd("IND", "Mumbai, Delhi, Bhubaneswar");
```

How to access a ConcurrentDictionary <TKey, TValue> Collection in C#?

We can access the key/value pairs of the `ConcurrentDictionary<TKey, TValue>` collection in C# using two different ways. They are as follows:

Using Index to access Individual ConcurrentDictionary<TKey, TValue> Collection Elements in C#:

We can access the individual value of the `ConcurrentDictionary` collection in C# by using the indexer. In this case, we just need to specify the key in the index to get the value from the given dictionary, no need to specify the index. If the specified key is not present, then the compiler will throw an exception. The syntax is given below.

```
dictionaryCountries["UK"]  
dictionaryCountries["USA"]
```

Using For-Each loop to Access ConcurrentDictionary<TKey,TValue> Collection in C#:

We can access the key/value pairs of a `ConcurrentDictionary<TKey, TValue>` in C# using a for loop as follows.

```
for (int i = 0; i<dictionaryCountries.Count; i++)  
{  
    string key = dictionaryCountries.Keys.ElementAt(i);  
    string value = dictionaryCountries[key];  
}
```

Using For-Each loop to Access ConcurrentDictionary<TKey,TValue> Collection in C#:

We can also use a for-each loop to access the key/value pairs of a ConcurrentDictionary<TKey, TValue> in C# as follows.

```
foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
{
    Console.WriteLine($"Key:{ KVP.Key}, Value: { KVP.Value}");
}
```

In the loop instead of KeyValuePair<TKey, TValue>, we can also use var as follows.

```
foreach (var item in dictionaryCountries)
{
    Console.WriteLine($"Key:{ item.Key}, Value: { item.Value}");
}
```

Example to Understand How to Create a ConcurrentDictionary<TKey, TValue> Collection and Add Elements in C#:

For a better understanding of how to create a ConcurrentDictionary<TKey, TValue> Collection and how to add elements, and how to access the elements of a Concurrent Dictionary in C#, please have a look at the below example.

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
using System.Linq;
namespace ConcurrentCollections
{
    class Program
    {
        static void Main(string[] args)
        {
            ConcurrentDictionary<string, string> dictionaryCountries = new
ConcurrentDictionary<string, string>();
            dictionaryCountries.TryAdd("UK", "London, Manchester, Birmingham");
            dictionaryCountries.TryAdd("USA", "Chicago, New York, Washington");
            dictionaryCountries.TryAdd("IND", "Mumbai, Delhi, Bhubaneswar");
            //The following key will not be added as IND already exists
            //It will not throw exception, simply TryAdd method returns false
            bool IsDupliacteKeyAdded = dictionaryCountries.TryAdd("IND", "Mumk
Bhubaneswar");

            //Accessing Dictionary Elements using For Each Loop
            Console.WriteLine("Accessing ConcurrentDictionary Elements using For
Each Loop");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            //Or
            //foreach (var item in dictionaryCountries)
```

```

//{
//    Console.WriteLine($"Key:{ item.Key}, Value: { item.Value}");
//}
//Accessing Dictionary Elements using For Loop
Console.WriteLine("\nAccessing ConcurrentDictionary Elements using
Loop");
for (int i = 0; i < dictionaryCountries.Count; i++)
{
    string key = dictionaryCountries.Keys.ElementAt(i);
    string value = dictionaryCountries[key];
    Console.WriteLine($"Key:{key}, Value: {value}");
}
//Accessing Dictionary Elements using Index Keys
Console.WriteLine("\nAccessing ConcurrentDictionary Elements using
Keys");
Console.WriteLine($"Key: UK, Value: {dictionaryCountries["UK"]}");
Console.WriteLine($"Key: USA, Value: {dictionaryCountries["USA"]}");
Console.WriteLine($"Key: IND, Value: {dictionaryCountries["IND"]}");
Console.ReadKey();
}
}
}

```

Output:

```

Accessing ConcurrentDictionary Elements using For Each Loop
Key:USA, Value: Chicago, New York, Washington
Key:IND, Value: Mumbai, Delhi, Bhubaneswar
Key:UK, Value: London, Manchester, Birmingham

Accessing ConcurrentDictionary Elements using For Loop
Key:USA, Value: Chicago, New York, Washington
Key:IND, Value: Mumbai, Delhi, Bhubaneswar
Key:UK, Value: London, Manchester, Birmingham

Accessing ConcurrentDictionary Elements using Index Keys
Key: UK, Value: London, Manchester, Birmingham
Key: USA, Value: Chicago, New York, Washington
Key: IND, Value: Mumbai, Delhi, Bhubaneswar

```

How to Check the Availability of a Key/Value Pair in ConcurrentDictionary Collection in C#?

If you want to check whether a key/value pair exists or not in the ConcurrentDictionary collection, then you can use the following ContainsKey method of the ConcurrentDictionary Collection Class in C#.

1. **ContainsKey(TKey key):** The ContainsKey(TKey key) method of the ConcurrentDictionary class is used to check if the given key is present in the ConcurrentDictionary or not. The parameter key to

locating in the ConcurrentDictionary object. If the given key is present in the collection, then it will return true else it will return false. If the key is null, then it will throw System.ArgumentNullException.

Let us understand this with an example. The following example shows how to use the ContainsKey method of the ConcurrentDictionary Collection class in C#.

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace ConcurrentCollections
{
    class Program
    {
        static void Main(string[] args)
        {
            ConcurrentDictionary<string, string> dictionaryCountries = new
ConcurrentDictionary<string, string>();
            dictionaryCountries.TryAdd("UK", "London, Manchester, Birmingham");
            dictionaryCountries.TryAdd("USA", "Chicago, New York, Washington");
            dictionaryCountries.TryAdd("IND", "Mumbai, Delhi, Bhubaneswar");
            //Accessing Dictionary Elements using For Each Loop
            Console.WriteLine("Accessing ConcurrentDictionary Elements using For
Loop");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }

            //Checking the key using the ContainsKey method
            Console.WriteLine("\nIs USA Key Exists : " +
dictionaryCountries.ContainsKey("USA"));
            Console.WriteLine("\nIs SL Key Exists : " +
dictionaryCountries.ContainsKey("SL"));

            Console.ReadKey();
        }
    }
}
```

Output:

```
Accessing ConcurrentDictionary Elements using For Each Loop
Key:UK, Value: London, Manchester, Birmingham
Key:IND, Value: Mumbai, Delhi, Bhubaneswar
Key:USA, Value: Chicago, New York, Washington

Is USA Key Exists : True

Is SL Key Exists : False
```


How to Remove Elements from ConcurrentDictionary Collection in C#?

If you want to remove an element from the ConcurrentDictionary, then you can use the following Remove method of the ConcurrentDictionary collection class.

1. **TryRemove(TKey key, out TValue value):** This method attempts to remove and return the value that has the specified key from the ConcurrentDictionary. Here, the parameter key specifies the key of the element to remove and return. The parameter value contains the object that is removed from the ConcurrentDictionary or the default value of the TValue type if the key does not exist. This method returns true if the object was removed successfully; otherwise, false. If the key is null, then it will throw ArgumentNullException.

If you want to remove all the elements from the ConcurrentDictionary collection, then you need to use the following Clear method of the ConcurrentDictionary class in C#.

1. **Clear():** This method is used to remove all elements i.e. all the keys and values from the ConcurrentDictionary object.

For a better understanding of how to use the Remove and Clear method of the ConcurrentDictionary collection class, please have a look at the below example.

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace ConcurrentDictionaryCollectionDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            ConcurrentDictionary<string, string> dictionaryCountries = new
ConcurrentDictionary<string, string>();
            dictionaryCountries.TryAdd("UK", "London, Manchester, Birmingham");
            dictionaryCountries.TryAdd("USA", "Chicago, New York, Washington");
            dictionaryCountries.TryAdd("IND", "Mumbai, Delhi, Bhubaneswar");
            //All ConcurrentDictionary Elements
            Console.WriteLine("All ConcurrentDictionary Elements");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            //Removing Element using TryRemove Method
            Console.WriteLine("\nRemoving Element using TryRemove Method");
            string removedCountry = string.Empty;
            bool result = dictionaryCountries.TryRemove("USA", out removedCountry);
            Console.WriteLine($"Is USA Key Removed: {result}");
            Console.WriteLine($"Removed Value: {removedCountry}");
            //NZ key not exists, so it will return false and removedCountry will be
the default of string i.e. null
            result = dictionaryCountries.TryRemove("NZ", out removedCountry);
```

```

        Console.WriteLine($"Is NZ Key Removed: {result}");
        Console.WriteLine($"Removed Value: {removedCountry}");
        //ConcurrentDictionary Elements After Removing USA Key
        Console.WriteLine("\nConcurrentDictionary Elements After Removing
foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
{
    Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
}

// Remove all Elements from ConcurrentDictionary Using Clear method
dictionaryCountries.Clear();
Console.WriteLine($"ConcurrentDictionary Elements Count After Clear:
{dictionaryCountries.Count}");
Console.ReadKey();
    }
}
}

```

Output:

```

All ConcurrentDictionary Elements
Key:USA, Value: Chicago, New York, Washington
Key:IND, Value: Mumbai, Delhi, Bhubaneswar
Key:UK, Value: London, Manchester, Birmingham

Removing Element using TryRemove Method
Is USA Key Removed: True
Removed Value: Chicago, New York, Washington

Is NZ Key Removed: False
Removed Value:

ConcurrentDictionary Elements After Removing USA Key
Key:IND, Value: Mumbai, Delhi, Bhubaneswar
Key:UK, Value: London, Manchester, Birmingham

ConcurrentDictionary Elements Count After Clear: 0

```

ConcurrentDictionary Collection Class Important Methods in C#

The ConcurrentDictionary Collection Class in C# provides some important methods. Let us discuss them one by one with Examples.

TryUpdate Method of ConcurrentDictionary Collection Class in C#:

TryUpdate(TKey key, TValue newValue, TValue comparisonValue): This method is used to update the value associated with the key to newValue if the existing value with the key is equal to the comparison value. Here, the parameter key specifies the key of the value that is compared with the comparison Value and possibly replaced. The parameter newValue specifies the value that replaces the value of the element that has the specified key if the comparison results in equality. And the parameter comparisonValue specifies the value that is compared with the value of the element that has the specified key. It returns true if the value with the key was equal to comparisonValue and was replaced with newValue; otherwise, false. If the key is null, then it will throw ArgumentNullException.

For a better understanding TryUpdate method of ConcurrentDictionary class in C#, please have a look at the below example.

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace ConcurrentCollections
{
    class Program
    {
        static void Main(string[] args)
        {
            ConcurrentDictionary<string, string> dictionaryCountries = new
ConcurrentDictionary<string, string>();
            dictionaryCountries.TryAdd("UK", "United Kingdom");
            dictionaryCountries.TryAdd("USA", "United State of America");
            dictionaryCountries.TryAdd("IND", "India");
            //All ConcurrentDictionary Elements
            Console.WriteLine("All ConcurrentDictionary Elements");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            // Try to update the key UK with new value if the old value = United
            bool result1 = dictionaryCountries.TryUpdate("UK", "United Kingdom",
"United");
            Console.WriteLine($"\\nIs the key UK update with TryUpdate Method:
{result1}");
            Console.WriteLine($"key UK, Value: {dictionaryCountries["UK"]}");
            // Try to update the key IND with new value if the old value = India
            bool result2 = dictionaryCountries.TryUpdate("IND", "Hindustan", "India");
            Console.WriteLine($"\\nIs the key IND update with TryUpdate Method:
{result2}");
            Console.WriteLine($"key IND, Value: {dictionaryCountries["IND"]}");
            Console.ReadKey();
        }
    }
}
```

Output:

```

All ConcurrentDictionary Elements
Key:UK, Value: United Kingdom
Key:IND, Value: India
Key:USA, Value: United State of America

Is the key UK update with TryUpdate Method: False
key UK, Value: United Kingdom

Is the key IND update with TryUpdate Method: True
key IND, Value: Hindustan

```

AddOrUpdate Methods of ConcurrentDictionary Collection Class in C#:

If the key doesn't exist, the AddOrUpdate() will add it. If the key exists, then the AddOrUpdate() method overwrites it with the value returned by the passed in the updateValueFactory delegate. It passes the current value to the delegate, which enables you to calculate a new value based on the current value. In other words, if you want to update existing keys based on the current value, use AddOrUpdate(). If you want to just overwrite the existing keys, use the indexer. The ConcurrentDictionary Collection Class provides the following three overloaded versions of the AddOrUpdate Method.

AddOrUpdate<TArg>(TKey key, Func<TKey, TArg, TValue> addValueFactory, Func<TKey, TValue, TArg, TValue> updateValueFactory, TArg factoryArgument): This method Uses the specified functions and argument to add a key/value pair to the ConcurrentDictionary if the key does not already exist, or to update a key/value pair in the ConcurrentDictionary if the key already exists. It returns the new value for the key. This will either be the result of addValueFactory if the key was absent or the result of updateValueFactory if the key was present. The following are the parameters used in this method.

1. **key:** The key to be added or whose value should be updated.
2. **addValueFactory:** The function used to generate a value for an absent key.
3. **updateValueFactory:** The function used to generate a new value for an existing key based on the key's existing value.
4. **factoryArgument:** An argument to pass into addValueFactory and updateValueFactory.
5. Type parameters: **TArg:** The type of an argument to pass into addValueFactory and updateValueFactory.

AddOrUpdate(TKey key, Func<TKey, TValue> addValueFactory, Func<TKey, TValue, TValue> updateValueFactory): This method uses the specified functions to add a key/value pair to the ConcurrentDictionary if the key does not already exist, or to update a key/value pair in the ConcurrentDictionary if the key already exists. It returns the new value for the key. This will either be the result of addValueFactory if the key was absent or the result of updateValueFactory if the key was present. The following are the parameters used in this method.

1. **key:** The key to be added or whose value should be updated.
2. **addValueFactory:** The function used to generate a value for an absent key.
3. **updateValueFactory:** The function used to generate a new value for an existing key based on the key's existing value.

AddOrUpdate(TKey key, TValue addValue, Func<TKey, TValue, TValue> updateValueFactory): This method adds a key/value pair to the ConcurrentDictionary if the key does not already exist, or updates a key/value pair in the ConcurrentDictionary by using the specified function if the key already exists. It returns

the new value for the key. This will either be `addValue` if the key was absent or the result of `updateValueFactory` if the key was present. The following are the parameters used in this method.

1. **key**: The key to be added or whose value should be updated.
2. **addValueFactory**: The function used to generate a value for an absent key.
3. **updateValueFactory**: The function used to generate a new value for an existing key based on the key's existing value.

Let us see an example to understand the above methods. In the first `AddOrUpdate` method the key `UK` exists in the dictionary `Countries`, so it will update the value. In the second `AddOrUpdate` method the key `SL` does not exist in the dictionary `Countries`, so it will add the key.

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace ConcurrentCollections
{
    class Program
    {
        static void Main(string[] args)
        {
            ConcurrentDictionary<string, string> dictionaryCountries = new
ConcurrentDictionary<string, string>();
            dictionaryCountries.TryAdd("UK", "United Kingdom");
            dictionaryCountries.TryAdd("USA", "United State of America");
            dictionaryCountries.TryAdd("IND", "India");
            //All ConcurrentDictionary Elements
            Console.WriteLine("All ConcurrentDictionary Elements");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            dictionaryCountries.AddOrUpdate("UK", "Kingdom United", (k, v) =>
Kingdom Updated");
            dictionaryCountries.AddOrUpdate("SL", "Srilanka", (k, v) => "Srilanka
Updated");
            Console.WriteLine("\nAll ConcurrentDictionary Elements After AddOrUpdate
Method");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:

```
All ConcurrentDictionary Elements
Key:UK, Value: United Kingdom
Key:IND, Value: India
Key:USA, Value: United State of America

All ConcurrentDictionary Elements After AddOrUpdate Method
Key:SL, Value: Srilanka
Key:UK, Value: United Kingdom Updated
Key:IND, Value: India
Key:USA, Value: United State of America
```

GetOrAdd Methods of ConcurrentDictionary Collection Class in C#

The GetOrAdd method of ConcurrentDictionary Class in C# is the same as AddOrUpdate except it will not change the existing value: it will only return it. Again, the ConcurrentDictionary Class provides three different overloaded versions of this method. They are as follows:

GetOrAdd(TKey key, Func<TKey, TValue> valueFactory): This method is used to add a key/value pair to the ConcurrentDictionary by using the specified function if the key does not already exist. Returns the new value, or the existing value if the key exists. The following are the parameters used in this method.

1. **key:** The key of the element to add.
2. **valueFactory:** The function used to generate a value for the key.

Returns: It Returns the value for the key. This will be either the existing value for the key if the key is already in the dictionary, or the new value if the key was not in the dictionary.

GetOrAdd(TKey key, TValue value): This method is used to add a key/value pair to the ConcurrentDictionary if the key does not already exist. Returns the new value, or the existing value if the key exists. The following are the parameters used in this method.

1. **key:** The key of the element to add.
2. **value:** The value to be added, if the key does not already exist.

Returns: It returns the value for the key. This will be either the existing value for the key if the key is already in the dictionary, or the new value if the key was not in the dictionary.

GetOrAdd<TArg>(TKey key, Func<TKey, TArg, TValue> valueFactory, TArg factoryArgument): This method is used to add a key/value pair to the ConcurrentDictionary by using the specified function and an argument if the key does not already exist, or returns the existing value if the key exists. The following are the parameters used in this method.

1. **key:** The key of the element to add.
2. **valueFactory:** The function used to generate a value for the key.
3. **factoryArgument:** An argument value to pass into valueFactory.
4. Type parameters: **TArg:** The type of an argument to pass into valueFactory.

Returns: It returns the value for the key. This will be either the existing value for the key if the key is already in the dictionary, or the new value if the key was not in the dictionary.

Let us see an example to understand the above methods. In the first GetOrAdd method the key UK exists in the dictionaryCountries, so it will simply retrieve the value and store it in the Result1 variable. In the second GetOrAdd method the key SL does not exist in the dictionaryCountries, so it will add the key SL with the value into the dictionaryCountries collection and then return the value.

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace ConcurrentCollections
{
    class Program
    {
        static void Main(string[] args)
        {
            ConcurrentDictionary<string, string> dictionaryCountries = new
ConcurrentDictionary<string, string>();
            dictionaryCountries.TryAdd("UK", "United Kingdom");
            dictionaryCountries.TryAdd("USA", "United State of America");
            dictionaryCountries.TryAdd("IND", "India");

            Console.WriteLine("ConcurrentDictionary Elements Before GetOrAdd Method");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            Console.WriteLine("\nGetOrAdd Method");
            // Get UK or add it with value of United Kingdom.
            string Result1 = dictionaryCountries.GetOrAdd("UK", "United Kingdom");
            Console.WriteLine($"Key:UK, Value: {Result1}");
            // Get SL or add it with value Srilanka.
            string Result2 = dictionaryCountries.GetOrAdd("SL", "Srilanka");
            Console.WriteLine($"Key:SL, Value: {Result2}");
            Console.WriteLine("\nConcurrentDictionary Elements After GetOrAdd Method");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            Console.ReadKey();
        }
    }
}
```

Output:


```
ConcurrentDictionary Elements Before GetOrAdd Method
```

```
Key:UK, Value: United Kingdom
```

```
Key:IND, Value: India
```

```
Key:USA, Value: United State of America
```

```
GetOrAdd Method
```

```
Key:UK, Value: United Kingdom
```

```
Key:SL, Value: Srilanka
```

```
ConcurrentDictionary Elements After GetOrAdd Method
```

```
Key:SL, Value: Srilanka
```

```
Key:UK, Value: United Kingdom
```

```
Key:IND, Value: India
```

```
Key:USA, Value: United State of America
```

TryGetValue Method of ConcurrentDictionary Collection Class in C#

TryGetValue(TKey key, out TValue value): This method attempt to get the value associated with the specified key from the ConcurrentDictionary. It returns true if the key was found in the ConcurrentDictionary, otherwise, false. The parameter key specifies the key of the value to get. The parameter value contains the object from the ConcurrentDictionary that has the specified key, or the default value of the type if the operation failed. So, if the key exists in the ConcurrentDictionary, then it will return true and the value with that associated key is stored on the value variable.

If you are not sure if a key is present or not in the ConcurrentDictionary, then you can use the TryGetValue() method to get the value from a ConcurrentDictionary because if you are not using TryGetValue then in that case you will get KeyNotFoundException. For a better understanding, please have a look at the below example.

```
using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace ConcurrentCollections
{
    class Program
    {
        static void Main(string[] args)
        {
            ConcurrentDictionary<string, string> dictionaryCountries = new
ConcurrentDictionary<string, string>();
            dictionaryCountries.TryAdd("UK", "United Kingdom");
            dictionaryCountries.TryAdd("USA", "United State of America");
            dictionaryCountries.TryAdd("IND", "India");

            Console.WriteLine("ConcurrentDictionary Elements Before GetOrAdd Method");
            foreach (KeyValuePair<string, string> KVP in dictionaryCountries)
            {
                Console.WriteLine($"Key:{KVP.Key}, Value: {KVP.Value}");
            }
            if(dictionaryCountries.TryGetValue("IND", out string Result1))
```

```

        {
            Console.WriteLine($"\\nKey = IND is found in the ConcurrentDicti
Value: {Result1}");
        }
        else
        {
            Console.WriteLine($"\\nKey = IND is not found in the
ConcurrentDictionary");
        }
        if (dictionaryCountries.TryGetValue("SL", out string Result2))
        {
            Console.WriteLine($"\\nKey = SL is found in the ConcurrentDicti
Value: {Result2}");
        }
        else
        {
            Console.WriteLine($"\\nKey = SL is not found in the
ConcurrentDictionary");
        }
        Console.ReadKey();
    }
}
}

```

Output:

```

ConcurrentDictionary Elements Before GetOrAdd Method
Key:UK, Value: United Kingdom
Key:IND, Value: India
Key:USA, Value: United State of America

Key = IND is found in the ConcurrentDictionary, Value: India

Key = SL is not found in the ConcurrentDictionary

```

ConcurrentDictionary Collection with Complex Type in C#:

As of now, we have used the built-in string and int types with ConcurrentDictionary. Now, let us proceed and see how to create a ConcurrentDictionary collection with Complex types. For this, let us create a class called Student. Then create a ConcurrentDictionary collection where the key is an integer which is nothing but the Id property of the student and the value is Student type. For a better understanding, please have a look at the below example.

```

using System;
using System.Collections.Concurrent;
using System.Collections.Generic;
namespace ConcurrentDictionaryCollections
{
    class Program

```

```

{
    static void Main(string[] args)
    {
        ConcurrentDictionary<int, Student> dictionaryStudents = new
ConcurrentDictionary<int, Student>();
        dictionaryStudents.TryAdd(101, new Student() { ID = 101, Name = "Anurag",
Branch = "CSE" });
        dictionaryStudents.TryAdd(102, new Student() { ID = 102, Name = "Mohanty",
Branch = "CSE" });
        dictionaryStudents.TryAdd(103, new Student() { ID = 103, Name = "Sambit",
Branch = "ETC" });

        Console.WriteLine("ConcurrentDictionary Elements");
        foreach (KeyValuePair<int, Student> item in dictionaryStudents)
        {
            Console.WriteLine($"Key: {item.Key}, ID: {item.Value.ID}, Name: {item.Value.Name}, Branch: {item.Value.Branch}");
        }

        Console.ReadKey();
    }
}
public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}
}

```

Output:

```

ConcurrentDictionary Elements
Key: 101, ID: 101, Name: Anurag, Branch: CSE
Key: 102, ID: 102, Name: Mohanty, Branch: CSE
Key: 103, ID: 103, Name: Sambit, Branch: ETC

```

How to Get all the Keys and Values of a ConcurrentDictionary in C#?

To get all the keys of the ConcurrentDictionary collection we have to use the Keys property of the ConcurrentDictionary class. Similarly, to get all the values of the ConcurrentDictionary collection, first, we need to get the keys, then we need to get the values using the keys. Even if you only want the values, then you can use the Values property of the ConcurrentDictionary collection class. For a better understanding, please have a look at the below example.

```

using System;
using System.Collections.Concurrent;
namespace ConcurrentDictionaryCollections
{

```

```

class Program
{
    static void Main(string[] args)
    {
        ConcurrentDictionary<int, Student> dictionaryStudents = new
ConcurrentDictionary<int, Student>();
        dictionaryStudents.TryAdd(101, new Student() { ID = 101, Name = "A",
Branch = "CSE" });
        dictionaryStudents.TryAdd(102, new Student() { ID = 102, Name = "Mc",
Branch = "CSE" });
        dictionaryStudents.TryAdd(103, new Student() { ID = 103, Name = "S",
Branch = "ETC" });
        //To get all the keys of ConcurrentDictionary use the keys property of
ConcurrentDictionary
        Console.WriteLine("All Keys in ConcurrentDictionary");
        foreach (int key in dictionaryStudents.Keys)
        {
            Console.WriteLine(key + " ");
        }
        // Once you get the keys, then get the values using the keys
        Console.WriteLine("\nAll Keys and values in ConcurrentDictionary");
        foreach (int key in dictionaryStudents.Keys)
        {
            var student = dictionaryStudents[key];
            Console.WriteLine($"Key: {key}, ID: {student.ID}, Name: {student.Name},
Branch: {student.Branch}");
        }
        //To get all the values in the ConcurrentDictionary use Values property of
ConcurrentDictionary
        Console.WriteLine("\nAll Student objects in ConcurrentDictionary");
        foreach (Student student in dictionaryStudents.Values)
        {
            Console.WriteLine($"ID: {student.ID}, Name: {student.Name}, Branch:
{student.Branch}");
        }
        Console.ReadKey();
    }
}

public class Student
{
    public int ID { get; set; }
    public string Name { get; set; }
    public string Branch { get; set; }
}

```

Output:

```
All Keys in ConcurrentDictionary
101
102
103

All Keys and values in ConcurrentDictionary
Key: 101, ID: 101, Name: Anurag, Branch: CSE
Key: 102, ID: 102, Name: Mohanty, Branch: CSE
Key: 103, ID: 103, Name: Sambit, Branch: ETC

All Student objects in ConcurrentDictionary
ID: 101, Name: Anurag, Branch: CSE
ID: 102, Name: Mohanty, Branch: CSE
ID: 103, Name: Sambit, Branch: ETC
```

In the next article, I am going to discuss the [ConcurrentQueue<T> Collection Class in C#](#) with Examples. Here, in this article, I try to explain the **ConcurrentDictionary<TKey, TValue> Collection Class in C#** with Examples. I hope this ConcurrentDictionary Collection Class in C# with Examples article will help you with your needs. I would like to have your feedback. Please post your feedback, question, or comments about this article.